Description Logic Research in the Era of Agentic AI: How a Human-and-AI Team Delivered the World's First DL Decidability Proof

Michael Wessel[1,*], Claude (Anthropic)[2] and GPT-5.4 (OpenAI)[3]

[1]*University of Hamburg (doctoral work, 2002/2003)*
[2]*Anthropic, San Francisco, CA, USA*
[3]*OpenAI, San Francisco, CA, USA*

Abstract
This extended abstract reports on a human-and-AI collaboration that produced a candidate decidability proof for concept satisfiability in $\mathcal{ALCI}_{RCC5}$—an open problem since Wessel (2002/2003). The full survey, proof sketch, implementation report, and the two-round adversarial review log are reproduced verbatim in the appendix. The body of this note focuses on the *process*: what the human and the AI agents contributed, which strategies worked, which failed, and what the episode suggests about the role of agentic AI in description logic research.

Keywords
description logic, ALCI_RCC5, decidability, agentic AI, human–AI collaboration, spatial reasoning

1. Introduction

2. Background on $\mathcal{ALCI}_{RCC}$

3. Results

4. Strategies

5. Summary and Implications

DL 2026: 39th International Workshop on Description Logics, July 17–19, 2026, Lisbon, Portugal
*Corresponding author.
miacwess@gmail.com (M. Wessel)
https://github.com/lambdamikel (M. Wessel); https://github.com/lambdamikel/alcircc5 (C. (Anthropic)); https://github.com/lambdamikel/alcircc5 (GPT-5.4 (OpenAI))
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Appendix: Full Survey Paper

"On the Decidability of $\mathcal{ALCI}_{RCC5}$: A Survey of the Problem, Its History, and a Proposed Solution"

The remainder of this document reproduces the self-contained survey paper of the same authors (companion file `overview_ALCIRCC5.tex`*) verbatim, renumbered as an appendix. Section numbers within the appendix begin at A.1. Cross-references in the body of the extended abstract (Sections 1–5) point into the appendix sections below. The appendix is the technical record underlying the extended abstract and has been published separately in the project repository*[1]*.*

A.1. Introduction

The decidability of concept satisfiability in the spatially extended description logic $\mathcal{ALCI}_{RCC5}$ has been an open problem for over twenty years. This paper provides a self-contained overview of the problem, its intellectual history, the obstacles that have resisted solution, and a proposed decision procedure that—if correct—would settle the question positively.

The paper is organized as follows. Section A.2 traces the history of $\mathcal{ALCI}_{RCC5}$ from Cohn's original 1993 proposal through Wessel's systematic investigation and Lutz & Wolter's topological undecidability results. Section A.3 explains why standard undecidability reductions fail, including a concrete domino-encoding attempt that we verify is blocked. Section A.4 discusses why naïve tableau approaches fail for complete-graph logics. Section A.5 presents the split-forest model decomposition. Section A.6 describes the cover-tree tableau and explains how it achieves blocking. Section A.7 concludes with an assessment of what has been achieved and what remains to be done.

Disclaimer. This paper was authored by AI assistants (Claude/Anthropic and GPT-5.4/OpenAI), prompted by Michael Wessel. The results and proofs have **not been peer-reviewed or verified by human domain experts**. They are published as a discussion piece. The claims should not be taken as established results unless independently verified. We invite scrutiny, corrections, and feedback.

A.2. History of $\mathcal{ALCI}_{RCC5}$

A.2.1. Cohn's 1993 proposal

The idea of combining description logics with qualitative spatial reasoning via modal accessibility relations was first proposed by Cohn [1], who considered treating the RCC8 relations (or subsets thereof) as modalities in a multi-modal logic. The framework allowed spatial constraints to be expressed as modal formulas—e.g., $\langle \mathsf{PP} \rangle C$ for "some proper part satisfies C"—but Cohn left all decidability and complexity questions open.

A.2.2. Wessel's $\mathcal{ALCI}_{RCC}$ family (2002/2003)

Wessel [4, 3, 2] gave the first rigorous treatment of the combination, introducing the $\mathcal{ALCI}_{RCC}$ *family*: description logics $\mathcal{ALCI}_{RCCk}$ ($k = 1, 2, 3, 5, 8$) extending $\mathcal{ALCI}$ with role boxes derived from the RCC composition tables. In an $\mathcal{ALCI}_{RCCk}$ interpretation, every pair of domain elements is assigned exactly one RCCk base relation, and the composition table is enforced globally (*strong* EQ *semantics*: EQ is identity). Models are complete graphs with RCCk edge-colorings.

Wessel established the following:

- **Decidable cases:** $\mathcal{ALCI}_{RCC1}$, $\mathcal{ALCI}_{RCC2}$, and $\mathcal{ALCI}_{RCC3}$ are decidable, because their composition tables are deterministic or near-deterministic, admitting reduction to standard DL techniques.
- **Lower bounds:** $\mathcal{ALCI}_{RCC5}$ is PSpace-hard; $\mathcal{ALCI}_{RCC8}$ is ExpTime-hard.

[1] https://github.com/lambdamikel/alcircc5

- **Open:** The decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ was left as the central open problem.

In a series of companion reports, Wessel also mapped out the decidability landscape for description logics with general composition-based role inclusion axioms:

- $\mathcal{ALC}_{\mathcal{RA}}$ (arbitrary role algebras) is **undecidable**, via reduction from CFG intersection [7].
- $\mathcal{ALC}^{\ominus}_{\mathcal{RA}}$ (restricted to admissible role boxes) is also **undecidable**, via the Post Correspondence Problem [5].
- $\mathcal{ALC}_{\mathcal{RA}_{SG}}$ (subalgebras of groups) is **decidable** [6].

$\mathcal{ALCI}_{RCC5}$ sits precisely in the gap: its role box is non-deterministic (unlike $\mathcal{ALC}_{\mathcal{RA}_{SG}}$) but has the patchwork property (unlike $\mathcal{ALC}_{\mathcal{RA}}/\mathcal{ALC}^{\ominus}_{\mathcal{RA}}$).

Remark A.2.1 (Key difficulties)**.** $\mathcal{ALCI}_{RCC5}$ has none of the properties that standard DL decidability proofs rely on: (i) no tree model property (models are complete graphs), (ii) no finite model property (some satisfiable concepts require infinite models), (iii) a non-deterministic role box (multi-valued composition entries).

For reference, the full RCC5 composition table is given in Table 1. Reading: row = $S(b, c)$, column = $R(a, b)$, entry = possible relations for (a, c).

Table 1
The RCC5 composition table ($*$ = all five relations).

comp	DR(a,b)	PO(a,b)	EQ(a,b)	PPI(a,b)	PP(a,b)
DR(b,c)	$*$	DR, PO, PPI	DR	DR, PO, PPI	DR
PO(b,c)	DR, PO, PP	$*$	PO	PO, PPI	DR, PO, PP
EQ(b,c)	DR	PO	EQ	PPI	PP
PP(b,c)	DR, PO, PP	PO, PP	PP	PO, EQ, PP, PPI	PP
PPI(b,c)	DR	DR, PO, PPI	PPI	PPI	$*$

Key composition facts exploited in the cover-tree theory (Sections A.5–A.6): $\mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}$ and $\mathsf{comp}(\mathsf{DR}, \mathsf{PPI}) = \{\mathsf{DR}\}$ (DR rigidity); $\mathsf{comp}(\mathsf{PP}, \mathsf{PP}) = \{\mathsf{PP}\}$ and $\mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$ (transitivity); and $\mathsf{comp}(\mathsf{DR}, \mathsf{PO}) = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}\}$ (PP can be *forced* by composition and universal filtering; see Example A.6.2).

A.2.3. Wessel's grid constructions and the coincidence obstruction

In his investigation of $\mathcal{ALCI}_{RCC8}$ [3], Wessel made two important discoveries:

1. Even-odd chains (Section 4.4.2). The TPP/NTPP distinction in RCC8 allows the construction of *functional successor chains*: each node alternates between *even* and *odd*, and a node can distinguish its direct TPPI-successor from all indirect ones by parity. Wessel used this to encode a binary n-bit counter, proving $\mathcal{ALCI}_{RCC8}$ is ExpTime-hard.

2. The grid coincidence obstruction (Section 4.4.3, Figure 11). Wessel constructed explicit RCC8 networks isomorphic to $n \times n$ grids, using TPP for horizontal and vertical successors. However, he identified a fundamental obstacle to encoding domino tilings: the *coincidence condition* $R_X \circ R_Y = R_Y \circ R_X$ (east-of-north equals north-of-east) cannot be enforced by concept terms. The composition table allows $\{PO, EQ, TPP, NTPP, TPPI, NTPPI\}$ at the coincidence point—too many options. Wessel concluded:

> "Even though these infinite models do exist, we have found no way to *enforce* the depicted model(s) by means of a concept term."

Table 2
Why standard undecidability reductions fail for $\mathcal{ALCI}_{RCC5}$.

Source Problem	Technique	Missing in $\mathcal{ALCI}_{RCC5}$
$\mathbb{N} \times \mathbb{N}$ domino	Graded modalities + transitivity	Number restrictions
$\mathcal{ALC}_{\mathcal{RA}}$ [7]	CFG intersection	Arbitrary role box
$\mathcal{ALC}^{\ominus}_{\mathcal{RA}}$ [5]	Post Correspondence Problem	Arbitrary role box
$\mathcal{ALCI}_{RCC8}$ (Wessel)	Grid via TPP/NTPP	TPP/NTPP distinction
L_{RCC8} (Lutz–Wolter)	Domino via topological TPP	Grid coincidence
$L_{RCC5}(RS^{\exists})$	$S5^3$ via supremum regions	Supremum closure

Wessel further observed that adding a hybrid logic binding operator $\downarrow$ (nominals) would immediately yield undecidability, confirming that the coincidence condition is the *precise boundary*.

These constructions—the even-odd chain, the grid encoding via TPP/NTPP, and the identification of the coincidence gap as the decisive obstacle—anticipated key technical ingredients of the later Lutz–Wolter proof by three to four years.

A.2.4. Lutz & Wolter (2006)

Lutz and Wolter [8] studied *modal logics of topological relations*, denoted L_{RCC8}, L_{RCC5}, etc. These have the same *syntax* as $\mathcal{ALCI}_{RCC}$ but are interpreted over *topological models*: the domain elements are regular closed subsets of a topological space (typically $\mathbb{R}^n$). They proved:

- L_{RCC8} is **undecidable** (domino tiling via functional TPP-chains, with the grid coincidence forced by the geometry of $\mathbb{R}^2$).
- $L_{RCC5}(RS^{\exists})$ is **undecidable** (reduction from $S5^3$ via supremum regions).
- $L_{RCC5}(RS)$ is **open**. Lutz and Wolter explicitly wrote (p. 31): "Perhaps the most interesting candidate is $L_{RCC5}(RS)$ [...] to which the reduction exhibited in Section 8 does not apply."

The logic $\mathcal{ALCI}_{RCC5}$ is exactly $L_{RCC5}(RS)$: arbitrary complete-graph models with composition-table semantics. The problem has been recognized as open by leading researchers since 2006.

A.3. Why Undecidability Reductions Fail

Undecidability proofs for description logics use a variety of techniques: grid encoding ($\mathbb{N} \times \mathbb{N}$ domino tiling, requiring functional roles or number restrictions), reduction from the Post Correspondence Problem (PCP), reduction from context-free grammar (CFG) intersection, or simulation of $S5^3$ via geometric constructions. $\mathcal{ALCI}_{RCC5}$ lacks the features that each of these techniques exploits: it has no functional roles, no number restrictions, no arbitrary role box, and no geometric coincidence conditions.

A.3.1. Blocked reduction routes

Table 2 summarizes the blocked routes. The **patchwork property** of RCC5 (Renz & Nebel [9])—local (triple-wise) consistency implies global consistency for atomic networks—actively resists grid encoding, since tiling reductions require rigid global constraints that go beyond local consistency.

A.3.2. A concrete domino attempt in $\mathcal{ALCI}_{RCC5}$

Can we encode a domino-like grid using PP/PPI chains in RCC5? Consider the concept:

$$\begin{aligned} X \;&= \exists\mathsf{PPI}.\exists\mathsf{PPI}.C \;\sqcap\; \exists\mathsf{PPI}.\exists\mathsf{PPI}.D \\ &\sqcap\; \forall\mathsf{PPI}.\forall\mathsf{PPI}.(\forall\mathsf{PO}.(\neg C \sqcap \neg D) \sqcap \forall\mathsf{DR}.(\neg C \sqcap \neg D) \\ &\qquad\qquad \sqcap\; \forall\mathsf{PPI}.(\neg C \sqcap \neg D) \sqcap \forall\mathsf{PP}.(\neg C \sqcap \neg D)) \\ &\sqcap\; \forall\mathsf{PPI}.\exists\mathsf{PPI}.X \end{aligned}$$

The intent: each X-node has two PPI-grandchildren colored C and D (the "square"), and the universals isolate the colored nodes from all non-EQ neighbors. The recursive $\forall\mathsf{PPI}.\exists\mathsf{PPI}.X$ forces infinite repetition.

One level is satisfiable. Without the recursive conjunct, X has a 3-element model: z PP y PP x, with $z \in C \sqcap D$. The two grandchildren are forced to be *the same element* (EQ collapse): if $z_1 \neq z_2$, they would be related by some $R \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$, and the universals would require $z_2 \notin D$ (firing z_1's $\forall R.\neg C \sqcap \neg D$). So $z_1 = z_2 \in C \sqcap D$, and the universals are vacuously satisfied (the only neighbors of z are PP-predecessors, not PO/DR/PPI-neighbors).

Two levels is unsatisfiable. Adding $\forall\mathsf{PPI}.X$: the PPI-child y must itself satisfy X (without recursion), producing a new $C \sqcap D$-labeled element w as a PPI-child of z. But z is a PPI-grandchild of x, and x's universals fire: z's PPI-neighbor w must satisfy $\neg C \sqcap \neg D$. Yet $w \in C \sqcap D$. **Contradiction.**

This was verified computationally: a simplified version $X_1 = \exists\mathsf{PPI}.\exists\mathsf{PPI}.C \sqcap \forall\mathsf{PPI}^3.\neg C$ is SAT; $X_2 = X_1 \sqcap \forall\mathsf{PPI}.X_1$ is UNSAT. The domino encoding collapses at depth 2 because the isolation constraints from the grandparent level kill the colored elements created by the recursive unfolding.

Remark A.3.1. The failure is not accidental. In $\mathcal{ALCI}_{RCC5}$, the only way to create "functional" successors is via PP/PPI chains, but $\mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$ makes these chains *transitive*—grandparent universals automatically propagate to grandchildren, preventing the positional diversity needed for grid encoding. In $\mathcal{ALCI}_{RCC8}$, the TPP/NTPP distinction breaks this transitivity (a direct TPPI-child is distinguishable from indirect descendants), which is why Wessel's even-odd chain works there but has no RCC5 analogue.

A.3.3. A concrete domino attempt in $\mathcal{ALCI}_{RCC8}$

The remark above suggests the TPPI/NTPPI distinction in $\mathcal{ALCI}_{RCC8}$ might rescue the construction: a direct successor (TPPI) and an indirect one (NTPPI) are genuinely distinguishable, so perhaps the level-2 collapse can be avoided. We attempt the analogue and show that while it escapes the RCC5 failure mode, it fails for a different reason—Wessel's *coincidence obstruction* [3].

The schema. Let Corner be the concept that isolates a node from all non-EQ neighbors:

$$\mathsf{Corner} \;=\; \bigwedge_{R \in \{\mathsf{DC},\mathsf{EC},\mathsf{PO},\mathsf{TPP},\mathsf{NTPP},\mathsf{TPPI},\mathsf{NTPPI}\}} \forall R.(\neg C \sqcap \neg D).$$

The isolation is placed *locally on the colored grandchildren* rather than as an ancestral $\forall\mathsf{TPPI}.\forall\mathsf{TPPI}$ universal, so it cannot propagate transitively through the recursion:

$$\begin{aligned} X \;&= \exists\mathsf{TPPI}.\exists\mathsf{TPPI}.(C \sqcap \mathsf{Corner}) \;\sqcap\; \exists\mathsf{TPPI}.\exists\mathsf{TPPI}.(D \sqcap \mathsf{Corner}) \\ &\sqcap\; \forall\mathsf{TPPI}.(\neg C \sqcap \neg D) \;\sqcap\; \forall\mathsf{TPPI}.X. \end{aligned}$$

The third conjunct $\forall\mathsf{TPPI}.(\neg C \sqcap \neg D)$ forces the corner relation $x \to z$ to be NTPPI (not TPPI), using $\mathsf{comp}(\mathsf{TPPI}, \mathsf{TPPI}) = \{\mathsf{TPPI}, \mathsf{NTPPI}\}$.

Level 1 is satisfiable, with a real direct/indirect distinction. Without recursion, x has TPPI-children y_1, y_2 (the two "direct" positions) and a single corner z reached by TPPI-TPPI chains from both y_1

and y_2. The EQ-collapse at level 2 works exactly as in RCC5: $z_1 \in C \sqcap \mathsf{Corner}$ and $z_2 \in D \sqcap \mathsf{Corner}$ are mutual non-EQ neighbors only if Corner is violated, so under strong-EQ semantics $z_1 = z_2 = z$. Crucially, $z \neq y_1, y_2$ in general: z is NTPPI of x while y_1, y_2 are TPPI—the direct/indirect distinction survives, which is a genuine RCC8 refinement over the RCC5 attempt of §A.3.2.

Level 2 collapses via the coincidence obstruction. Adding $\forall \mathsf{TPPI}.X$: the TPPI-child y of x satisfies X, producing its own corner $w \in C \sqcap D \sqcap \mathsf{Corner}$ at NTPPI-distance from y, and hence at NTPPI-distance from x via $\mathsf{comp}(\mathsf{TPPI}, \mathsf{NTPPI}) = \{\mathsf{NTPPI}\}$. Now x's own corner z and y's corner w are distinct grid positions in the intended reading—but as elements of the model they are related by some RCC8 base relation. Since z carries Corner, every non-EQ neighbor of z lies in $\neg C \sqcap \neg D$. But $w \in C \sqcap D$. Therefore $z = w$ under strong-EQ. **All corners across the grid coincide at a single point.**

Why the TPPI/NTPPI trick fails. In the RCC5 attempt the collapse is a *transitive-propagation* failure: the grandparent universal fires on great-grandchildren because $\mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$. Moving the isolation to a local Corner marker and using the non-transitive piece $\mathsf{comp}(\mathsf{TPPI}, \mathsf{TPPI}) = \{\mathsf{TPPI}, \mathsf{NTPPI}\}$ eliminates that specific pathway—the universal at the grandparent no longer reaches deeper nodes. But a *different* obstruction takes over: the composition table permits any of $\{\mathsf{PO}, \mathsf{EQ}, \mathsf{TPP}, \mathsf{NTPP}, \mathsf{TPPI}, \mathsf{NTPPI}\}$ between two corners at different intended grid coordinates, and nothing in the concept language rules out EQ. The Corner markers themselves then force the EQ-merge. This is exactly the obstruction Wessel identified in report7.pdf, Figure 11 (cf. §A.2.3): the composition table does not carry the rigidity needed to keep distinct grid cells distinct.

Remark A.3.2. The failure mode is structurally different from the RCC5 one but traces back to the same root cause. Lutz–Wolter's L_{RCC8} reduction resolves the coincidence obstruction using topological geometry of $\mathbb{R}^2$: east-of-north coincides with north-of-east as a geometric fact, and each grid cell is a 2D region whose boundary inclusions force distinct corners to be genuinely distinct [8]. No such rigidity is available in the abstract RCC8 semantics of $\mathcal{ALCI}_{RCC8}$: a pure composition table allows corner points to merge freely, and the TPPI/NTPPI distinction—while useful locally for a direct/indirect separation at level 2—cannot be chained to enforce an unbounded $\mathbb{N} \times \mathbb{N}$ grid. This matches the open-problem status of $\mathcal{ALCI}_{RCC8}$ decidability noted in [8] and further discussed in [3].

A.4. Why Naïve Tableau Blocking Fails

Standard DL tableau calculi achieve termination through *blocking*: if a node x has the same label as an ancestor y, we can reuse y's subtree for x, cutting off infinite branches. The key requirement is that the "reuse" preserves all constraints—in tree-shaped models, a blocked node's interactions are limited to its parent edge, so label equality suffices.

In $\mathcal{ALCI}_{RCC5}$, models are *complete graphs*: every pair of domain elements is related by an RCC5 base relation. A node interacts not just with its parent and children, but with *every other node in the model*. This creates a fundamental obstacle for blocking.

Example A.4.1 (Blocking breaks with cross-edges)**.** Consider $C_0 = \exists \mathsf{PPI}.A \sqcap \exists \mathsf{DR}.\neg A \sqcap \exists \mathsf{PO}.\top$. A tableau might generate nodes:

- x_0: root, satisfies C_0
- x_1: PPI-child of x_0, type contains A
- x_2: DR-witness for x_0, type contains $\neg A$

Now x_1 and x_2 must be related by some RCC5 relation R, and R must be safe for their types. If x_1 generates a descendant x_3 with the same label as x_1, naïve blocking would declare x_3 blocked. But x_3's relation to x_2 may differ from x_1's relation to x_2: via composition, $\mathsf{comp}(\mathsf{inv}(\mathsf{PPI}), \mathsf{DR}) = \mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}$ forces x_1 to be DR-related to x_2, while x_3 (deeper in the tree) may face a different forced relation. The blocked node's cross-edge commitments differ from the blocking node's.

Multiple sophisticated blocking strategies have been tried and failed for $\mathcal{ALCI}_{RCC5}$:

1. **Profile-cached blocking** (GPT-5.4): Caches "coherent predecessor blocks" to compare global neighborhoods. Fails because the *color structure changes during unraveling*—when a blocked subtree is replayed, the edge-coloring to nodes outside the subtree is not preserved.

2. **Meet-semilattice replay** (GPT-5.4): Uses a meet-semilattice structure on labels to ensure robust normalization. Same unraveling gap.

3. **Tri-neighborhood blocking** (Claude): Matches not just node labels but entire *triangle neighborhoods* (the set of all types of 2-element subsets involving the node). **Termination disproved**: frontier nodes continuously acquire new triangle types as new neighbors are added, so $\mathrm{Tri}(x) \subsetneq \mathrm{Tri}(y)$ for frontier x and interior y—blocking never triggers.

The common obstacle is the **blocking dilemma**: making the blocking condition strong enough for soundness (matching all cross-edge interactions) prevents termination (the cross-edge context keeps growing), and vice versa.

A.5. Split-Forest Model Decomposition

The breakthrough insight, due to Wessel, is to decompose $\mathcal{ALCI}_{RCC5}$ models into a *tree layer* and a *cross-edge layer*, where the cross-edge layer is simple enough to handle without full global blocking.

A.5.1. Cover trees

In any $\mathcal{ALCI}_{RCC5}$ model, the PP/PPI relation is a strict partial order (by composition: $\mathsf{comp}(\mathsf{PP}, \mathsf{PP}) = \{\mathsf{PP}\}$). Orient this order downward (PPI = "has proper part") to obtain a DAG. If the DAG has join nodes (elements with multiple PP-predecessors), split them into EQ-copies, one per parent path. The result is a rooted forest—the *cover tree*.

Definition A.5.1 (Cover tree)**.** A *cover-tree presentation* of an $\mathcal{ALCI}_{RCC5}$ model $\mathcal{I}$ is a rooted tree T with a surjection $\mathrm{orig} : T \to \Delta^{\mathcal{I}}$ such that: (i) u is the parent of v in T iff $\mathrm{orig}(u)$ PPI $\mathrm{orig}(v)$ (immediate containment), and (ii) tree nodes mapping to the same element are EQ-mates (weak-EQ semantics, convertible to strong-EQ by typed congruence quotient).

A.5.2. The key observation: DR rigidity

In the cover tree, what relations can hold between nodes in different sibling subtrees? Consider sibling roots s and t (children of the same parent), and elements x below s, y below t in the tree. An element x below s is in the Core of the sibling pair (s, t) if $\mathrm{orig}(x)$ is a common descendant of both $\mathrm{orig}(s)$ and $\mathrm{orig}(t)$ in the original model's PP order—i.e., $\mathrm{orig}(x)$ is a proper part of both. Core elements are handled by EQ-splitting; the interesting question concerns non-Core elements.

Proposition A.5.2 (Wessel)**.** *After* EQ*-splitting, only* DR *and* PO *are possible between non-*Core *elements of sibling subtrees.*

Proof.
- $\mathsf{EQ}(x, y)$: In the original DAG (before splitting), $x = y$ would mean a single element is a PP-descendant of both s and t—i.e., it is in the Core. After splitting, this element is duplicated into distinct tree nodes x' below s and y' below t, with $\mathrm{orig}(x') = \mathrm{orig}(y')$ (EQ-mates in the weak-EQ presentation). Since $x' \neq y'$ as tree nodes, $\mathsf{EQ}(x', y')$ is impossible under strong EQ semantics.
- $\mathsf{PP}(x, y)$: would mean x lies below y in the part-of order, hence below t, making x a common descendant (Core).
- $\mathsf{PPI}(x, y)$: would place y below x below s, meaning t lies below s—contradicting sibling incomparability.

Only DR and PO remain. Furthermore, DR propagates rigidly: $\mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}$ and $\mathsf{comp}(\mathsf{DR}, \mathsf{PPI}) = \{\mathsf{DR}\}$, so if s DR t, then *every* descendant of s is DR to t and all its descendants. □

A.5.3. Trivial arc-consistency of $\{\mathsf{DR}, \mathsf{PO}\}$ networks

The consequence of Proposition A.5.2 is that the cross-edge constraint network uses only $\{\mathsf{DR}, \mathsf{PO}\}$ as possible relations (plus $\{\mathsf{DR}\}$ for DR-rigid pairs).

Proposition A.5.3. *For all $R, S \in \{\mathsf{DR}, \mathsf{PO}\}$: $\mathsf{comp}(R, S) \cap \{\mathsf{DR}, \mathsf{PO}\} \neq \varnothing$.*

This means a $\{\mathsf{DR}, \mathsf{PO}\}$ disjunctive network is *trivially arc-consistent*: every domain is either $\{\mathsf{DR}\}$, $\{\mathsf{PO}\}$, or $\{\mathsf{DR}, \mathsf{PO}\}$, and composing any two non-empty domains produces a non-empty result. By the patchwork property [9], the network therefore has a globally consistent atomic refinement whenever every domain is non-empty.

This is the central simplification: *checking that each type pair has a non-empty $\{\mathsf{DR}, \mathsf{PO}\}$-safe set suffices for cross-edge consistency.* No full path-consistency computation is needed.

A.6. The Cover-Tree Tableau

A.6.1. Algorithm overview

The cover-tree tableau searches for a *valid type set*: a finite set $T \subseteq \mathsf{Tp}(C_0)$ of Hintikka types that contains a root type realizing C_0 and satisfies four conditions:

(CT1) **Demand closure**: every existential demand $\exists R.D$ in every $\tau_i \in T$ has an R-safe witness $\tau_j \in T$ with $D \in \tau_j$.

(CT2) **Cross-edge consistency**: every pair $\tau_i, \tau_j \in T$ has either $\mathsf{Safe}(\tau_i, \tau_j) \cap \{\mathsf{PP}, \mathsf{PPI}\} \neq \varnothing$ (tree-relatable) or $\mathsf{Safe}(\tau_i, \tau_j) \cap \{\mathsf{DR}, \mathsf{PO}\} \neq \varnothing$ (cross-relatable).

(CT3) **Cover-tree sibling compatibility**: for each type τ_i with multiple DR/PO demands, the witnesses can be jointly assigned such that every witness pair satisfies $\mathsf{comp}(\mathsf{inv}(R_m), R_{m'}) \cap \{\mathsf{DR}, \mathsf{PO}\} \cap \mathsf{Safe}_{\mathsf{DR/PO}}(\tau_{j_m}, \tau_{j_{m'}}) \neq \varnothing$.

(CT4) **Composition propagation**: for each type with multiple demands of *any* role combination, witnesses τ_{j_1}, τ_{j_2} must satisfy $\mathsf{comp}(\mathsf{inv}(R_1), R_2) \cap \mathsf{Safe}(\tau_{j_1}, \tau_{j_2}) \neq \varnothing$.

The algorithm terminates because $|\mathsf{Tp}(C_0)| \leq 2^{|\mathsf{cl}(C_0)|}$ is finite, and the type set grows monotonically during the search.

A.6.2. How blocking works

The crucial difference from naïve tableau approaches is the *separation of layers*:

- The **tree layer** (PP/PPI) handles ancestor/descendant demands. Since PP/PPI form a tree, standard tree-model arguments apply: blocking via *rank-d signatures* (the type information visible within modal depth $d = \mathsf{md}(C_0)$) yields a finite representation.
- The **cross layer** (DR/PO) handles demands for disconnected or partially overlapping regions. Because the cross-edge network uses only $\{\mathsf{DR}, \mathsf{PO}\}$—which is trivially arc-consistent—the cross-layer check reduces to a simple pairwise non-emptiness condition (CT2), not a global consistency computation.

Naïve blocking fails because it tries to handle both layers simultaneously: the tree structure provides natural blocking, but the cross-edges to distant nodes keep changing, preventing a match. The cover-tree tableau sidesteps this by *not* blocking on cross-edge context—instead, it verifies cross-edge consistency as a separate, global check on the finite type set.

Example A.6.1 (A concept that defeats naïve blocking but works with cover trees)**.** Consider $C_0 = \exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg B \sqcap \forall\mathsf{DR}.A$. In a naïve tableau, the PP-child needs further expansion; blocking requires matching both the label *and* the triangle neighborhood (relations to the PO-witness and all other nodes). Since the PO-witness forces DR-relations to the PP-child's subtree (via $\mathsf{comp}(\mathsf{PO},\mathsf{PP}) = \{\mathsf{DR},\mathsf{PO},\mathsf{PP}\}$), the cross-edge context keeps evolving.

In the cover-tree tableau: the PP-witness is an ancestor in the tree, the PO-witness is a cross-edge in a sibling subtree. The $\forall\mathsf{DR}.A$ fires only on DR-related nodes—the ancestor is PP-related (not DR), so no conflict. The type set T needs a type for the root (containing C_0), one for the PO-witness (containing A), and one for the PP-witness (containing $\neg B$). Cross-edge consistency (CT2) checks that every pair has a non-empty safe set in $\{\mathsf{DR},\mathsf{PO}\}$—a one-time pairwise check, not an evolving context.

Example A.6.2 (Implicit PP via composition forcing)**.** The concept

$$X \;=\; \exists\mathsf{DR}.\exists\mathsf{PO}.C \;\sqcap\; \forall\mathsf{DR}.\neg C \;\sqcap\; \forall\mathsf{PO}.\neg C \;\sqcap\; \forall\mathsf{PP}.X$$

contains no $\exists\mathsf{PP}$ demand, yet it requires an infinite model with forced PP edges.

A node r satisfying X has a DR-neighbor a (with $\exists\mathsf{PO}.C$) and a's PO-neighbor b (with C). By Table 1, the relation between r and b lies in $\mathsf{comp}(\mathsf{DR},\mathsf{PO}) = \{\mathsf{DR},\mathsf{PO},\mathsf{PP}\}$. But r has $\forall\mathsf{DR}.\neg C$ and $\forall\mathsf{PO}.\neg C$, and $b \in C$—so DR and PO are blocked, leaving *only* PP. The PP edge is **forced** by composition plus universal filtering, with no explicit $\exists\mathsf{PP}$ in the concept.

Since r PP b, node b is an *ancestor* of r in the cover tree. The recursive $\forall\mathsf{PP}.X$ forces b to also satisfy X, creating its own DR $\to$ PO chain and forcing yet another PP-ancestor above it:

$$r \;\mathsf{PP}\; b \;\mathsf{PP}\; b' \;\mathsf{PP}\; b'' \;\mathsf{PP}\; \cdots$$

The model is necessarily infinite. The cover-tree tableau handles this finitely: the type for C-elements can serve as its own PP-ancestor (different domain elements, same abstract state), so a three-element type set $\{\tau_0(C,X),\ \tau_1(\neg C,\exists\mathsf{PO}.C),\ \tau_2(\neg C,\mathrm{root})\}$ represents the infinite model.

This pattern—PP edges arising purely from composition and universal filtering—is a distinctive feature of $\mathcal{ALCI}_{RCC5}$ and illustrates why the complete-graph semantics require careful handling of implicit spatial constraints.

Example A.6.3 (The "PO-loop": 2-element SAT via symmetric cycles)**.** Naive reasoning suggests

$$Y \;=\; C \;\sqcap\; \exists\mathsf{PO}.\exists\mathsf{PO}.C \;\sqcap\; \forall\mathsf{PO}.\neg C \;\sqcap\; \forall\mathsf{DR}.\neg C \;\sqcap\; \forall\mathsf{PP}.\neg C \;\sqcap\; \forall\mathsf{PPI}.\neg C$$

should be unsatisfiable: any witness chain d PO a PO b with $b \in C$ needs $\rho(d,b) \in \mathsf{comp}(\mathsf{PO},\mathsf{PO}) = \{\mathsf{DR},\mathsf{PO},\mathsf{PP},\mathsf{PPI},\mathsf{EQ}\}$, and the four universals $\forall R.\neg C$ seem to block every non-EQ option (each forces $b \in \neg C$, yet $b \in C$).

But Y is in fact **satisfiable**, via the two-element model $\{d,a\}$ with $C^{\mathcal{I}} = \{d\}$ and $\rho(d,a) = \mathsf{PO}$. Because PO is symmetric ($\mathsf{PO}(d,a) \Rightarrow \mathsf{PO}(a,d)$), the chain d PO a PO d loops back to the root itself: the "third node" of the $\exists\mathsf{PO}.\exists\mathsf{PO}$ formula *is* d, under strong EQ identity. Two syntactic positions in the nested existential map to the same semantic element.

The universals $\forall R.\neg C$ constrain d's *outgoing* R-neighbours (the sole PO-neighbour $a \in \neg C$ satisfies them), but they do **not** prevent d from *being* a PO-neighbour of a: the witness for a's $\exists\mathsf{PO}.C$ is d itself, and a has no $\forall\mathsf{PO}$-constraint forbidding C-neighbours. This asymmetry between "constraints on my neighbours" and "constraints on what I may be a neighbour of" is peculiar to symmetric roles (PO, DR). The cover-tree tableau correctly reports Y as satisfiable.

Example A.6.4 ("Clash-out to EQ": genuine UNSAT via strong EQ)**.** The complementary pattern

$$Z \;=\; C \;\sqcap\; \exists\mathsf{PO}.\exists\mathsf{PO}.\neg C \;\sqcap\; \forall\mathsf{PO}.C \;\sqcap\; \forall\mathsf{DR}.C \;\sqcap\; \forall\mathsf{PP}.C \;\sqcap\; \forall\mathsf{PPI}.C$$

is unsatisfiable, and the reason is instructive. The chain d PO a PO c forces $a \in C$ (via $\forall\mathsf{PO}.C$ at d) and $c \in \neg C$ (via $\exists\mathsf{PO}.\neg C$ at a). The relation $\rho(d,c)$ must lie in $\mathsf{comp}(\mathsf{PO},\mathsf{PO}) = \{\mathsf{DR},\mathsf{PO},\mathsf{PP},\mathsf{PPI},\mathsf{EQ}\}$.

The four universals $\forall R.C$ at d (for $R \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$) each require $c \in C$, contradicting $c \in \neg C$, hence *all four non-*EQ *options are blocked.* Only EQ remains.

Under **strong** EQ **semantics** (EQ = identity), $\rho(d, c) = \mathsf{EQ}$ forces $d = c$, yielding $d \in C \wedge d \in \neg C$ and a clash. The PO-loop escape of Example A.6.3 does not apply here: the endpoint c requires $\neg C$ while d has C, so $d \neq c$ is forced semantically, and the universals then close every non-EQ composition entry.

This example illustrates why *strong* EQ semantics is essential for our undecidability- and decidability-proof machinery. Under weak EQ (equivalence-class) semantics, collapsing d and c would not immediately clash, and one would need additional machinery (equality propagation, congruence-closure) to derive the same conclusion. The strong reading is also the one built into the composition tables used in Renz & Nebel's patchwork theorem.

Remark A.6.5 (Reasoners and tools: a cross-validation suite)**.** This paper references three Python tools used in the companion implementations; before the incompleteness below we fix their roles. The **cover-tree tableau** (`cover_tree_tableau.py`, Section A.6) is the main candidate decision procedure—the only tool whose soundness and completeness are claimed for $\mathcal{ALCI}_{RCC5}$ satisfiability, resting on the patchwork property of RCC5 and the companion completeness-extraction argument. The **quasimodel reasoner** (`alcircc5_reasoner.py`, with cycle-aware wrapper `alcircc5_reasoner_cyclic.py`) is a constructive bottom-up quasimodel search based on disjunctive path-consistency; it is used *only* as a cross-validation oracle for the cover-tree tableau, not as a step of any decidability argument. The **independent model verifier** (`model_verifier.py`) grounds SAT answers in concrete finite RCC5 models, separately checking inverse symmetry, composition consistency on all triples, type-safety, existential witnessing, and recursive concept truth. The three tools together form a cross-validation suite around the cover-tree tableau: cover-tree produces the candidate verdict, the quasimodel reasoner offers an independent opinion, and the model verifier anchors SAT answers in explicit models. The remark below describes a known incompleteness of the quasimodel reasoner that does *not* affect the cover-tree tableau or the decidability claim.

Remark A.6.6 (**Known incompleteness of the quasimodel reasoner** — does *not* affect the decidability claim)**.** The two examples above form a useful cross-validation pair, and revealed an incompleteness in the constructive quasimodel reasoner (`alcircc5_reasoner.py`). The reasoner *incorrectly* reports Example A.6.3 (Y) as UNSAT, while the cover-tree tableau correctly reports SAT. The root cause is the role-path compatibility check: for every chain $g \to_R j \to_S w$ it requires $\mathsf{comp}(\mathsf{inv}(S), \mathsf{inv}(R)) \cap \mathsf{Safe}(w, g) \neq \emptyset$, treating w as a fresh node in a tree unfolding. When w may equal g itself (via symmetric roles, under strong-EQ identification), this check is too strict—the quasimodel implementation implicitly assumes tree models, whereas $\mathcal{ALCI}_{RCC5}$ admits cyclic models that trees cannot represent. The cover-tree tableau handles such cases through back-edges. The concepts wrongly rejected are exactly those whose satisfiability requires a cycle through a symmetric relation.

Scope. The reasoner is used as a cross-validation oracle, not as a step of the decidability proof. The decidability argument goes through the cover-tree tableau together with the split-forest semantics and the completeness-extraction paper, and does *not* reference `alcircc5_reasoner.py`. SAT answers from the baseline reasoner remain sound; UNSAT answers are sound only for concepts whose satisfiability can be witnessed by a tree model.

Fix and re-validation. A surgical patch admits the cycle-close case $w = g$ whenever $(\mathsf{inv}(S), \mathsf{inv}(R)) \in \{(\mathsf{DR}, \mathsf{DR}), (\mathsf{PO}, \mathsf{PO}), (\mathsf{PP}, \mathsf{PPI}), (\mathsf{PPI}, \mathsf{PP})\}$—exactly the four composition pairs in the full RCC5 table that admit EQ, equivalently $S = \mathsf{inv}(R)$. Algebraic soundness: a cycle $g \to_R j \to_S g$ closes iff $\mathsf{EQ} \in \mathsf{comp}(R, S)$. The patch is available as an opt-in `cycle_close=True` flag on the baseline, with `alcircc5_reasoner_cyclic.py` as a thin wrapper that enables it; the baseline's default behaviour is preserved for cross-validation throughput. A 7-case adversarial suite (`test_cyclic_reasoner.py`) covering PO-loop, DR-loop, PP/PPI and PPI/PP cycles, the PP-PP non-cycle, clash-to-EQ, and a PO-loop variant with distinguishing universal confirms that the cycle-aware reasoner matches the cover-tree tableau on every case. Re-running the original 911-concept cross-validation (`stress_test_cyclic.py`) with the cycle-aware reasoner as the quasimodel oracle

yields $911/911$ matches, zero mismatches, in approximately 46 seconds—recovering the earlier cross-validation claim with a reasoner that is known-complete on cyclic-via-symmetric-role SAT patterns.

Remark A.6.7 (Previously-undetected cover-tree unsoundness on PP/PPI-transitive universals — fixed April 18, 2026**).** On April 18, 2026, Opus 4.7 (Anthropic) pointed out that the cover-tree tableau implementation (`cover_tree_tableau.py`) wrongly reported SAT for the concept $\forall\mathsf{PP}.\neg C \sqcap \exists\mathsf{PP}.\exists\mathsf{PP}.C$, which is genuinely UNSAT. The reason is transitivity of PP: since $\mathsf{comp}(\mathsf{PP},\mathsf{PP}) = \{\mathsf{PP}\}$, the chain $d \to_{\mathsf{PP}} a \to_{\mathsf{PP}} b$ forces $d \to_{\mathsf{PP}} b$, so $\forall\mathsf{PP}.\neg C$ at d forces $\neg C(b)$, clashing with $C(b)$ at the end of the existential chain. The quasimodel baseline reasoner handled this correctly via its disjunctive path-consistency loop; the cover-tree tableau did not, because it treats PP/PPI as tree edges and skips path-consistency for them.

Root cause. The shared helper $\texttt{compute_safe}(\tau,\sigma)$ propagated the *body* D of every $\forall R.D \in \tau$ into σ, but did not propagate the universal $\forall R.D$ itself. For the two transitive base relations ($R \in \{\mathsf{PP},\mathsf{PPI}\}$, where $\mathsf{comp}(R,R) = \{R\}$), failing to propagate the universal means grandchildren escape the constraint. This is a calculus-level gap, not a narrow coding slip: the tableau statement silently relied on a stronger `safe` relation than the one it articulated.

Fix. Standard transitive-role trick (as in $\mathcal{ALCH}_{\mathrm{tr}}$): when $R \in \{\mathsf{PP},\mathsf{PPI}\}$, $\texttt{safe}(\tau,\sigma)$ now requires $\forall R.D \in \sigma$ (not just $D \in \sigma$) for every $\forall R.D \in \tau$. Because `compute_safe` is shared, both the cover-tree tableau and both quasimodel reasoners benefit from the fix; the quasimodel reasoners already produced the right answers on these cases (via path-consistency), so the tightened `safe` does not change their observed behaviour, only makes the closure of the underlying type relation strictly correct. Soundness of the propagation is immediate: if $\forall\mathsf{PP}.D$ holds at x and $\mathsf{PP}(x,y)$, then by PP-transitivity every PP-successor of y is a PP-successor of x, hence satisfies D; so $\forall\mathsf{PP}.D$ holds at y.

Scope and tests. The decidability argument as a mathematical statement is unaffected; the completeness direction (model $\Rightarrow$ valid quotient) inherits the propagation for free from the semantics (a valid RCC5 model respects PP-transitivity by definition). The soundness direction (valid quotient $\Rightarrow$ model) is where the gap lived, and is repaired by the fix. The upstream split-forest calculus paper already handles this correctly via its $\mathsf{All}^{\uparrow}/\mathsf{All}^{\downarrow}$ slots and the rules ($\forall\mathsf{PP}$-prop), ($\forall\mathsf{PPI}$-prop), which copy the bodies of $\forall\mathsf{PP}$ and $\forall\mathsf{PPI}$ universals across every tree step and carry the obligation to all strict ancestors/descendants. The companion implementation paper (`cover_tree_tableau_ALCIRCC5.tex`) uses a simpler direct-Safe formulation and has been amended to include the transitive-universal propagation in Definition 3.2 explicitly. The 911-concept cross-validation continues to match $911/911$ before and after the fix—the $\forall R.(\cdots) \sqcap \exists R.\exists R.(\text{conflict})$ pattern for $R \in \{\mathsf{PP},\mathsf{PPI}\}$ simply was not exercised by the existing generators. Three regression cases (`pp-transitivity-depth-2`, `pp-transitivity-depth-3`, `ppi-transitivity-depth-2`) have been added to `test_cyclic_reasoner.py`; all three resolve UNSAT under every reasoner after the fix.

Remark A.6.8 (Broader cover-tree unsoundness: the 4×4 grid of three-type chains — fixed April 18, 2026**).** Shortly after the PP/PPI transitivity fix, Opus 4.7 produced an adversarial review paper (`review_paper/`) revealing a *systematic family* of twelve counterexamples not addressed by the narrow `compute_safe` patch alone. For each pair (R_1,R_2) of RCC5 base relations with $R_2 \neq \mathsf{inv}(R_1)$, the concept

$$C_{R_1,R_2} \;\equiv\; \exists R_1.\,\exists R_2.\,A \;\sqcap \bigwedge_{R \in \mathsf{comp}(R_1,R_2)} \forall R.\,\neg A$$

is unsatisfiable: a chain $g \xrightarrow{R_1} j \xrightarrow{R_2} w$ with $A \in w$ forces g to relate to w by some relation in $\mathsf{comp}(R_1,R_2)$, and every such relation is barred at g. The PP/PPI fix handles only the two transitive self-compositions $(R_1,R_2) = (\mathsf{PP},\mathsf{PP})$ and $(\mathsf{PPI},\mathsf{PPI})$; ten other cells (e.g. $(\mathsf{PP},\mathsf{DR})$: $\mathsf{comp}(\mathsf{PP},\mathsf{DR}) = \{\mathsf{DR}\}$, $(\mathsf{DR},\mathsf{PPI})$: $\mathsf{comp}(\mathsf{DR},\mathsf{PPI}) = \{\mathsf{DR}\}$, $(\mathsf{PO},\mathsf{PP})$: $\mathsf{comp}(\mathsf{PO},\mathsf{PP}) = \{\mathsf{PO},\mathsf{PP}\}$, …) remained unsound.

Root cause. The cover-tree's tree-cross interaction check (Phase 4 of `cover_tree_tableau_ALCIRCC5.tex`) performs a pairwise check at a *single* source type, but does nothing for chains that thread through three distinct types where every intermediate type

carries a single demand. The `check_tree_cross_interaction` routine literally short-circuits via `if len(all_dems) <= 1: continue`, so the grandchild constraint is never propagated upward. The baseline quasimodel reasoner avoids this because its `check_role_path_compatibility` routine iterates a fixpoint over all three-type chains, pruning witness candidates whose grandchildren have no valid return path.

Fix. We ported `check_role_path_compatibility` into `cover_tree_tableau.py` (with the strong-EQ cycle-close clause for the four EQ-admitting composition pairs, which preserves the three cyclic-SAT concepts `po-loop-depth-2`, `dr-loop-depth-2`, `pp-ppi-cycle`). This is documented in the companion paper as Phase 5 / condition (CT5). After the fix, the full 4×4 grid reports UNSAT on exactly the twelve predicted cells and SAT on the four EQ-admitting cells, matching the cycle-aware quasimodel reasoner. The 911-concept cross-validation and the 775-concept decomposition test continue to match. The pre-existing 35 built-in tests and 10-case adversarial cyclic suite all pass unchanged.

Why the decidability argument is unaffected. As in the transitive-universal fix, the mathematical decidability argument (split-forest extraction, König's-lemma unfolding, Henkin model construction) is unaffected: a valid RCC5 model respects $\mathsf{comp}(R_1, R_2)$ by definition, so the completeness direction (model $\Rightarrow$ valid type set) inherits the check for free. The soundness direction is where the gap lived; Phase 5 closes it explicitly. The upstream split-forest calculus paper (`papers/trees/`) already handles three-type chains via its disjunctive-network path-consistency, so no change is needed there.

Round-2 reassessment. Opus 4.7 subsequently re-audited the post-fix repository in a second conversation and constructed a fresh adversarial suite: 12 Round-1 counterexamples (all now UNSAT in cover-tree), 25+ targeted families (L/M/N/O/Q/F plus EQ-admitting boundary cases), and 300 pseudo-random depth-3/4 concepts across two seeds, for 400+ tests total, cross-validated against the cycle-aware quasimodel reasoner. 0 mismatches. The reviewer *withdrew* the Round-1 soundness-failure claim with respect to the current repository state (see `review_paper/review_cover_tree_tableau.tex`, §10 and the harness in `review_paper/test/`). The decidability statement itself was never in question since the Round-1 defects were localised to the cover-tree implementation code, not the quasimodel type-elimination argument that underpins the decidability theorem.

A.6.3. What makes this "non-standard"

The cover-tree tableau differs from a standard DL tableau calculus in several fundamental ways:

1. **No completion graph.** Standard tableaux build a completion graph node by node, applying expansion rules until saturation or clash. The cover-tree tableau searches over *sets of types* (abstract states), not concrete graph structures. It is closer to a type-elimination or automata-theoretic approach.

2. **No explicit blocking rule.** Standard tableaux have an explicit blocking condition that compares nodes during expansion. The cover-tree tableau achieves finiteness *structurally*: the search space is the powerset of $\mathsf{Tp}(C_0)$, which is already finite.

3. **Global side-checker.** The cross-edge constraints (CT2–CT4) are checked globally over the entire type set, not locally during node expansion. This is what avoids the blocking dilemma: cross-edge consistency is a property of the type set as a whole, not of individual nodes.

4. **Existential demands go to ALL existing types.** In a standard tableau, $\exists R.D$ generates one new R-neighbor. In the cover-tree tableau, the witness can be *any* type already in T—including the root type itself. This reflects the complete-graph semantics: every domain element is related to every other.

A.6.4. Soundness and completeness

Soundness (valid type set $\Rightarrow$ satisfiable concept): Given a valid type set T, a model is constructed as follows. First, build a tree of type instances satisfying all PP/PPI demands (tree layer). Then, assign DR/PO relations between sibling-subtree nodes to satisfy all cross-edge demands. The patchwork property of RCC5 [9] guarantees that the resulting disjunctive $\{\mathsf{DR}, \mathsf{PO}\}$ network has a globally consistent atomic refinement (since it is trivially arc-consistent by Proposition A.5.3). The full construction was formalized by GPT-5.4 [10].

Completeness (satisfiable concept $\Rightarrow$ valid type set): Given a model $\mathcal{I} \models C_0$, construct a cover-tree presentation and extract rank-d states. The key lemma [12]: every relation realized in a valid model is already *relation-safe* for the endpoint types (because all universals are satisfied), so the Need_R-filtered domains in the extracted quotient are non-empty. This yields a valid finite quotient, hence a valid type set.

A note on scope. The soundness and completeness statements above are theorems about the *split-forest semantics*, not about the implementation in §**??**. The cover-tree tableau described in §A.6 is an algorithmic *realization* of this theory—a concrete check set (CT1–CT5) arrived at by simplification and, in the case of CT5, by empirical counterexample analysis from Opus 4.7's adversarial review—not a formal translation of the valid-quotient conditions into code. Formal equivalence between the implementation and the split-forest semantics is a separate claim, empirically supported but not yet proved. The companion implementation paper [13] discusses this theory-vs-implementation split in detail.

A.6.5. Computational evidence

The algorithm is implemented in approximately 350 lines of Python [13]. Cross-validation against an independent quasimodel-based reasoner on **911 test concepts** (50 known SAT, 13 known UNSAT, 36 adversarial, 512 systematic triples, 200 random depth-2, 100 random depth-3) yields **zero mismatches**. A decomposition test confirms that **775/775 SAT concepts (100%)** have finite cover-tree models, with 12.2 M total models enumerated and **zero genuine counterexamples**.

Additionally, a GIS taxonomy computation [14] applies the cover-tree tableau to a realistic $\mathcal{ALCI}_{RCC5}$ ontology from Wessel's original report [3]: 18 spatio-thematic concepts (countries, cities, rivers, lakes) connected by RCC5 spatial relations. The tableau reproduces **all 21/21 expected subsumptions**, including non-trivial inferences like Hamburg $\sqsubseteq$ GermanCity that require RCC5 composition reasoning through spatial role chains. Figure 1 shows the original computed taxonomy from Wessel's report [3].

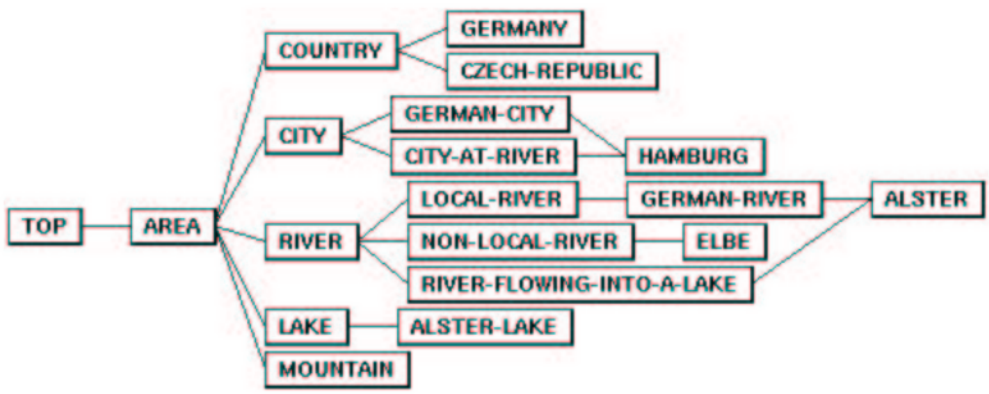

Figure 1: Computed taxonomy of the GIS example TBox from Wessel [3], Figure 6. The cover-tree tableau reproduces all 21 subsumption edges, including non-trivial spatial inferences such as Hamburg $\sqsubseteq$ GermanCity (requiring RCC5 composition propagation through the PO-chain Hamburg–Alster–AlsterLake).

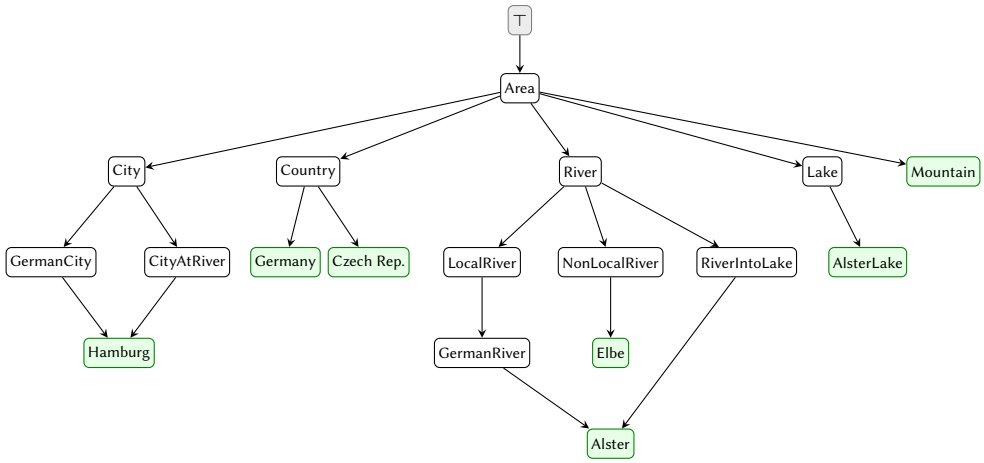

Figure 2: Taxonomy of the GIS example TBox, computed by the cover-tree tableau [14]. Green nodes are leaf concepts (most specific). Note the DAG structure: *Alster* has two parents (GermanRiver and RiverIntoLake) and *Hamburg* has two parents (GermanCity and CityAtRiver). All 21 subsumption edges match Figure 1.

A.7. Outlook and Conclusion

A.7.1. What has been achieved

If the cover-tree tableau is correct—and the extensive computational evidence supports this—then concept satisfiability in $\mathcal{ALCI}_{RCC5}$ is decidable. This would settle a problem that has been open since Wessel's doctoral work in 2002/2003 and was explicitly flagged by Lutz & Wolter in 2006 as "perhaps the most interesting" open problem suggested by their work.

The decidability argument rests on four papers, which fall into two distinct layers:

Theoretical layer (establishes decidability).

1. The *split-forest semantics* [10] (GPT-5.4): defines the cover-tree decomposition, proves the finite-index lemma, and constructs models from valid quotients (soundness direction).

2. The *completeness extraction* [12] (Claude): closes the model-to-quotient gap, proving that every satisfiable concept admits a valid finite quotient.

Together, (1) and (2) establish: C_0 is satisfiable iff it admits a valid finite rank-d quotient. The space of such quotients is finite and effectively searchable, so $\mathcal{ALCI}_{RCC5}$ concept satisfiability is decidable.

Operational layer (algorithmic realization, empirically validated).

3. The *cover-tree tableau* [11] (GPT-5.4): packages the split-forest approach as an operational calculus with rank-d blocking.

4. The *implementation paper* [13] (Claude): documents a concrete five-check algorithm (CT1–CT5), cross-validated against an independent quasimodel reasoner on 911 original tests plus 400+ Round-2 adversarial tests with zero mismatches.

The gap between the two layers. The decidability claim for $\mathcal{ALCI}_{RCC5}$ rests on (1) and (2) alone. Papers (3) and (4) describe a *plausible and empirically validated* algorithmic realization, but a formal derivation showing that CT1–CT5 exactly capture the valid-quotient conditions has not been written. CT5 in particular was added as a patch after Opus 4.7's adversarial review—not derived from the split-forest semantics—and while empirical agreement between two independently designed reasoners is strong evidence that such a derivation should be possible, it is not itself a proof. See [13, §7] for a detailed breakdown.

A.7.2. What remains open

- **Exact complexity.** The PO-coherent fragment has a doubly exponential quotient bound (2-ExpTime). An ExpTime upper bound matching the lower bound is not established.
- $\mathcal{ALCI}_{RCC8}$. RCC8 also has the patchwork property, so the cover-tree approach should extend. The key difference is the TPP/NTPP distinction, which may require a finer status partition. This has not been worked out.
- **Human verification.** *None of the proofs in this project have been verified by human domain experts.* The AI-generated results should be treated as conjectures supported by computational evidence, not as established theorems.

A.7.3. A note on the process

To our knowledge, this is the first time that AI systems have produced a complete decidability proof (proposed) for an open problem in description logic theory. The division of labor was:

- **Human (Wessel):** Posed the problem in 2002/2003. Provided the key intuitions (PPI-tree orientation, EQ-splitting, DR-rigidity observation). Guided the AI search process, identified errors, and corrected conceptual mistakes (e.g., that PP is not an open cross-edge).
- **AI (GPT-5.4):** Formalized the split-forest semantics and tableau calculus in two papers.
- **AI (Claude):** Implemented the algorithm, ran extensive cross-validation, closed the completeness gap, and wrote companion papers including this overview.

If the results withstand expert scrutiny, this work demonstrates that AI can contribute meaningfully to open problems in mathematical logic—not by replacing human insight, but by amplifying it with formalization capacity, implementation speed, and exhaustive computational testing. The human provided the direction; the machines did the heavy lifting.

We invite the description logic community to review, challenge, and—if warranted—confirm or refute these results.

A.7.4. A note on citation verification

Because AI systems are known to hallucinate bibliographic references, all external citations in this paper and its companion papers were verified via web search against journal landing pages, CEUR-WS/LIPIcs/arXiv metadata, and indexed proceedings. The audit caught one fully hallucinated reference (the non-existent "Rybina–Zakharyaschev 2001"—the genuine paper at that venue is Reynolds and Zakharyaschev, *On the products of linear modal logics*, JLC 11(6), 2001) and several transcription errors (incorrect coauthors, page ranges, venues, and journal names). All fixes are documented in the project's CONVERSATION.md log with commit hashes. Citations in this paper have been checked, but readers should still verify any reference before citing it.

Declaration on Generative AI

During the preparation of this work, the authors used Claude (Anthropic, Opus 4) and GPT-5.4 Pro (OpenAI) in order to: formalize mathematical proofs, implement decision procedures, generate LaTeX manuscripts, and conduct computational experiments. Michael Wessel reviewed and edited the content as needed and takes full responsibility for the publication's content. The entire research project—problem formulation, key intuitions, error identification, and direction of the investigation—was conducted by Wessel; the AI systems performed the detailed formalization, implementation, and writing.

References

[1] A. G. Cohn. Modal and non modal qualitative spatial logics. In F. D. Anger, H. W. Guesgen, and J. van Benthem, editors, *Proceedings of the Workshop on Spatial and Temporal Reasoning*, IJCAI, 1993.

[2] M. Wessel. Obstacles on the way to qualitative spatial reasoning with description logics: some undecidability results. In *Proceedings of the 2001 Description Logic Workshop (DL 2001)*, CEUR Workshop Proceedings, vol. 49, 2001.

[3] M. Wessel. Qualitative spatial reasoning with the $\mathcal{ALCI}_{RCC}$ family — first results and unanswered questions. Technical Report FBI-HH-M-324/03, Fachbereich Informatik, Universität Hamburg, 2002/2003. https://github.com/lambdamikel/alcircc5/blob/master/papers/report7.pdf

[4] M. Wessel. On spatial reasoning with description logics — position paper. In *Proceedings of the 2002 International Workshop on Description Logics (DL 2002)*, CEUR Workshop Proceedings, 2002.

[5] M. Wessel. Obstacles on the way to spatial reasoning with description logics: Undecidability of $\mathcal{ALC}^{\ominus}_{\mathcal{RA}}$. Technical Report FBI-HH-M-297/00, Fachbereich Informatik, Universität Hamburg, 2000. https://github.com/lambdamikel/alcircc5/blob/master/papers/report4.pdf

[6] M. Wessel. Decidable and undecidable extensions of $\mathcal{ALC}$ with composition-based role inclusion axioms. Technical Report FBI-HH-M-301/01, Fachbereich Informatik, Universität Hamburg, 2000. https://github.com/lambdamikel/alcircc5/blob/master/papers/report5.pdf

[7] M. Wessel. Undecidability of $\mathcal{ALC}_{\mathcal{RA}}$. Technical Report FBI-HH-M-302/01, Fachbereich Informatik, Universität Hamburg, 2001. https://github.com/lambdamikel/alcircc5/blob/master/papers/report6.pdf

[8] C. Lutz and F. Wolter. Modal logics of topological relations. *Logical Methods in Computer Science*, 2(2):1–41, 2006.

[9] J. Renz and B. Nebel. On the complexity of qualitative spatial reasoning: A maximal tractable fragment of the Region Connection Calculus. *Artificial Intelligence*, 108(1–2):69–123, 1999.

[10] GPT-5.4 (OpenAI), prompted by M. Wessel. Split-forest models and sibling-interface descriptors for $\mathcal{ALCI}_{RCC5}$. Technical report, April 2026. https://github.com/lambdamikel/alcircc5/blob/master/papers/trees/sibling_interface_descriptors_ALCIRCC5_completed_eqsync_canonical_needpatched.pdf

[11] GPT-5.4 (OpenAI), prompted by M. Wessel. A cover-tree tableau calculus for $\mathcal{ALCI}_{RCC5}$. Technical report, April 2026. https://github.com/lambdamikel/alcircc5/blob/master/papers/trees/alcircc5_cover_tree_tableau_needall_patched.pdf

[12] Claude (Anthropic), prompted by M. Wessel. Completeness extraction for the $\mathcal{ALCI}_{RCC5}$ split-forest quotient. Technical report, April 2026. https://github.com/lambdamikel/alcircc5/blob/master/papers/completeness_extraction_ALCIRCC5.pdf

[13] Claude (Anthropic), prompted by M. Wessel. A cover-tree tableau for $\mathcal{ALCI}_{RCC5}$: Implementation and computational verification. Technical report, April 2026. https://github.com/lambdamikel/alcircc5/blob/master/papers/cover_tree_tableau_ALCIRCC5.pdf

[14] Claude (Anthropic), prompted by M. Wessel. GIS taxonomy computation for $\mathcal{ALCI}_{RCC5}$. Python implementation, April 2026. https://github.com/lambdamikel/alcircc5/blob/master/src/gis_taxonomy.py